

Symbolic derivations behind the paper

This file re-derives, in a computer algebra system, every analytical step of *Wave-driven propulsion of a flexible raft*. Each printed relation is obtained from the equations it is claimed to follow from, and is then compared with what the paper prints. Nothing is assumed: where the paper asserts that a term vanishes, the term is constructed and shown to vanish.

Structural claims carry a negative control. A check that passes for the right reason must fail when a sign or a factor is changed, so each such claim is paired with a deliberately broken variant that has to fail. Without the control a passing test says nothing.

Equation numbers refer to the submitted manuscript. The sections below follow the paper in order: the governing system of Section 2, the thrust and asymmetry diagnostics, the reduced-order modal model, and Appendices A to C.

Coverage. The file certifies Sections 2.2 to 2.6 and Appendices A, B and C. It does not cover the numerical discretisation, which the rest of the test suite exercises, nor the Longuet-Higgins prefactor of (2.30), which is taken from the cited literature.

```
using Test
using LinearAlgebra
using Random
using Symbolics

Random.seed!(20260922)
setprecision(BigFloat, 220)
```

Working with the imaginary unit

Symbolics simplifies expressions over `Complex{Num}` unreliably, so the imaginary unit is carried as an ordinary symbol `Iu` and reduced by hand through the rules below. Viscous terms enter as `eps = 1/Re`, and the paper retains only first order in viscosity, so higher powers of `eps` are dropped.

```
@variables Iu eps

const IRULES = Dict{Iu^2 => -1, Iu^3 => -Iu, Iu^4 => 1, Iu^-1 => -Iu}
const TRUNC = Dict{eps^2 => 0, eps^3 => 0, eps^4 => 0}

"Reduce powers of the imaginary unit and truncate at first order in 1/Re."
function redI(e)
    e = expand(e)
    for _ in 1:8
        e = expand(substitute(e, IRULES))
    end
    for _ in 1:4
        e = expand(substitute(e, TRUNC))
    end
    Symbolics.simplify(e)
end

iszero0(x) = (v = Symbolics.value(x); v isa Number ? iszero(v) : isequal(v, 0))

"True when the expression reduces to zero."
zz(e) = iszero0(redI(e)) ||
        iszero0(Symbolics.simplify_fractions(redI(e))) ||
        iszero0(Symbolics.simplify_fractions(e))
```

```
"True when an expression in ordinary symbols reduces to zero."
zp(e) = (v = Symbolics.value(Symbolics.simplify(expand(e), expand = true));
        v isa Number ? isapprox(Float64(real(v)), 0.0; atol = 1e-13) : isequal(v, 0))
```

Exact integration of a polynomial over the raft, used wherever the paper projects onto the modal basis.

```
@variables x

function antiderivative(p)
    p = expand(p)
    s = 0
    for k in 0:14
        c = k == 0 ? substitute(p, Dict(x => 0)) : Symbolics.coeff(p, x^k)
        s += c * x^(k + 1) / (k + 1)
    end
    s
end

raftint(p) = Symbolics.simplify(substitute(antiderivative(p), Dict(x => 1 // 2)) -
                                substitute(antiderivative(p), Dict(x => -1 // 2)))

D1 = Differential(x)
dx(e, k = 1) = (for _ in 1:k
                e = Symbolics.expand_derivatives(D1(e))
                end; e)
at(e, p) = Symbolics.simplify(substitute(e, Dict(x => p)))
```

Section 2.4: the harmonic operators

The paper defines the operator of the kinematic condition and its approximate inverse. The inverse is exact only to first order in viscosity, and the residual of the product fixes the order of the error.

```
@testset "2.4 the operator K and its approximate inverse" begin
    @variables a
    ## Kbar = i - 2a, Kbarinv = -i - 2a with a standing for (2/Re) d_xx acting on
    ## a single Fourier mode. Their product differs from unity at second order.
    prod = redI((Iu - a) * (-Iu - a))
    @test zz(prod - (1 + a^2))
    ## The error term is +a^2, not -a^2. A sign error here would be invisible in
    ## the leading behaviour, so it is pinned separately.
    @test !zz(prod - (1 - a^2))
end

@testset "2.4 the free-surface row (2.26b)" begin
    ## Towers of x-derivatives, closed under d_xx up to the truncation order.
    @variables Gam Fr2 Wen
    @variables fb fb_2 fb_4 fb_6 fz fz_2 fz_4 fz_6 eb eb_2 eb_4 eb_6
    DXX = Dict(fb => fb_2, fb_2 => fb_4, fb_4 => fb_6, fb_6 => 0,
              fz => fz_2, fz_2 => fz_4, fz_4 => fz_6, fz_6 => 0,
              eb => eb_2, eb_2 => eb_4, eb_4 => eb_6, eb_6 => 0)
    Kb(e) = redI(substitute(e, Dict(u => Iu * u - 2 * eps * DXX[u] for u in keys(DXX))))
    Kbinv(e) = redI(substitute(e, Dict(u => -Iu * u - 2 * eps * DXX[u] for u in keys(DXX))))

    @test zz(Kb(Kbinv(fz)) - fz)
    @test zz(Kbinv(Kb(fz)) - fz)
```

```

## The modal dynamic pressure of Section 2.6, written with Laplace already
## applied as in the manuscript, p_dyn = -(i phi - (2/Re) phi_xx).
pdyn = -(Iu * fb - 2 * eps * fb_2)
## Total pressure, p = Gamma (p_dyn - eta / Fr^2), with eta = Kinv phi_z.
ptot = redI(Gam * (pdyn - Kbinv(fz) / Fr2))
## The capillary condition off the raft, p = -eta_xx / We.
resid = redI(ptot + Kbinv(fz_2) / Wen)
cleared = Kb(resid)

printed = fb + 4 * Iu * eps * fb_2 - fz / Fr2 + fz_2 / (Gam * Wen)
factor = redI(Symbolics.coeff(cleared, fb) / Symbolics.coeff(redI(printed), fb))
@test zz(cleared - factor * printed)

## Controls: flipping either constituent breaks the row.
@test !zz(Kb(redI(Gam * (-pdyn - Kbinv(fz) / Fr2) + Kbinv(fz_2) / Wen)) - factor * printed)
@test !zz(Kb(redI(ptot - Kbinv(fz_2) / Wen)) - factor * printed)
end

```

Section 2.5: mean thrust and radiation asymmetry

Splitting the boundary elevations into symmetric and antisymmetric parts turns the difference of outgoing intensities into a single interference term. Both the thrust numerator and the asymmetry factor follow from it.

```

@testset "2.5 the S and A decomposition (2.32) and alpha (2.31)" begin
  @variables Sr Si Ar Ai
  sq(u, v) = u^2 + v^2
  right = sq(Sr + Ar, Si + Ai)
  left = sq(Sr - Ar, Si - Ai)
  ReSA = Sr * Ar + Si * Ai

  @test zp((left - right) + 4 * ReSA)
  @test zp((right + left) - 2 * (sq(Sr, Si) + sq(Ar, Ai)))
  @test zp((left - right) / (right + left) - (-2 * ReSA) / (sq(Sr, Si) + sq(Ar, Ai)))

  ## The three cases in which the radiation is symmetric and the thrust vanishes.
  @test zp(substitute(left - right, Dict{Ar => 0, Ai => 0}))
  @test zp(substitute(left - right, Dict{Sr => 0, Si => 0}))
  @test zp(substitute(left - right, Dict{Si => 0, Ar => 0}))
  ## A generic pair radiates asymmetrically, so the three cases above are not vacuous.
  @test !zp(left - right)
end

```

Section 2.6: the reduced-order modal model

The raft balance is non-dimensionalised with the scales of Section 2.5, then projected onto the modal basis. The projection produces the modal balance, and substituting the two linear maps produces the impedance matrix.

```

@testset "2.6 the non-dimensional raft balance (2.33)" begin
  @variables rho rhoR g om nu L dd EI etb pdynb fb
  @variables kap Lam Gam Fr2
  ## Dimensional harmonic beam equation with the hydrostatic part separated,
  ## EI eta'''' - rho_R om^2 eta + d rho g eta = d p_dyn - f.
  lhs = EI * (L * etb) / L^4 - rhoR * om^2 * (L * etb) + dd * rho * g * (L * etb)

```

```

## The dynamic pressure scale follows from the potential,  $p_{\text{dyn}} = \rho L^2 \omega^2 \bar{p}$ .
rhs = dd * (rho * L^2 * om^2 * pdynb) - rhoR * L * om^2 * fb
derived = expand((lhs - rhs) / (rhoR * L * om^2))

groups = Dict(kap => EI / (rhoR * L^4 * om^2), Lam => dd / L,
              Gam => rho * L^2 / rhoR, Fr2 => L * om^2 / g)
printed = kap * etb + (Lam * Gam / Fr2 - 1) * etb - (Lam * Gam * pdynb - fb)
@test zp(Symbolics.simplify_fractions(derived - expand(substitute(printed, groups))))

## Control: the raft pressure scale  $\rho R \omega^2$  would give  $\Lambda$ , not  $\Lambda \Gamma$ .
wrong = kap * etb + (Lam * Gam / Fr2 - 1) * etb - (Lam * pdynb - fb)
@test !zp(Symbolics.simplify_fractions(derived - expand(substitute(wrong, groups))))
end

@testset "2.6 modal balance (2.34) and impedance matrix (2.36)" begin
  @variables kap Lam Gam Fr2 Wen qm bm4 pm fm Km Zq Cq
  ## Projecting the raft balance leaves the capillary endpoint term on the left.
  projected = kap * bm4 * qm + (Lam / Wen) * Km + (Lam * Gam / Fr2 - 1) * qm -
              (Lam * Gam * pm - fm)
  printed = (kap * bm4 + Lam * Gam / Fr2 - 1) * qm -
              (Lam * Gam * pm - fm - (Lam / Wen) * Km)
  @test zp(projected - printed)

  ## Substituting  $p = Z q$  and  $K = C q$  and collecting gives  $M q = -f$ .
  Dmm = kap * bm4 + Lam * Gam / Fr2 - 1
  derivedM = Dmm * qm - Lam * Gam * Zq + (Lam / Wen) * Cq
  with_maps = Dmm * qm - (Lam * Gam * Zq - fm - (Lam / Wen) * Cq)
  @test zp(with_maps - (derivedM + fm))

  ## The capillary map enters with a plus sign. The manuscript prints
  ##  $M = D - \Lambda (\Gamma Z - C/We)$ , which is this expression.
  @test zp((Dmm * qm - Lam * (Gam * Zq - (1 / Wen) * Cq)) - derivedM)
  ## Control: the opposite sign on the capillary map does not reproduce it.
  @test !zp((Dmm * qm - Lam * (Gam * Zq + (1 / Wen) * Cq)) - derivedM)
end

@testset "2.6 parity of the modal maps" begin
  ##  $Z_{mn}$  is the projection of a load of parity  $(-1)^n$  onto a mode of parity
  ##  $(-1)^m$ . Opposite parities integrate to zero over the centred raft.
  @variables c1 c2 c3 g1 g2 g3
  parity_fun(k, cs) = sum(cs[j] * x^(2 * (j - 1) + (k % 2)) for j in 1:length(cs))
  for (m, n) in ((0, 0), (0, 1), (1, 0), (1, 1), (2, 2), (2, 3), (3, 3))
    Z = raftint(parity_fun(m, (c1, c2, c3)) * parity_fun(n, (g1, g2, g3)))
    if (m + n) % 2 == 1
      @test zp(Z)
    else
      @test !zp(Z)
    end
  end
end

## The capillary map inherits the same structure through the two edge slopes,
## since reflection exchanges them with the sign of the mode parity.
@variables Wp sp
for (m, n) in ((0, 1), (1, 0), (2, 3), (0, 2), (1, 3))

```

```

    C = Wp * sp + (-1)^m * Wp * (-1)^n * sp
    @test zp(C - ((m + n) % 2 == 1) ? 0 : 2 * Wp * sp))
end
end

@testset "2.6 the far-field amplitudes split by parity" begin
    @variables a0 a1 a2 a3 q0 q1 q2 q3
    ## Reflection gives  $a_n(-l) = (-1)^n a_n(l)$ , so the even modes carry S and the
    ## odd modes carry A.
    right = a0 * q0 + a1 * q1 + a2 * q2 + a3 * q3
    left = a0 * q0 - a1 * q1 + a2 * q2 - a3 * q3
    @test zp((right + left) / 2 - (a0 * q0 + a2 * q2))
    @test zp((right - left) / 2 - (a1 * q1 + a3 * q3))
    @test zp(substitute(right - left, Dict{q1 => 0, q3 => 0}))
    @test zp(substitute(right + left, Dict{q0 => 0, q2 => 0}))
end

```

Appendix B.1: the free-free modal basis

The paper writes the modes about the centre of the raft, while the textbook form is written on $[0, L]$. The two are the same functions, and the two half-angle conditions of the paper factor the textbook frequency equation.

```

const TOLF = 1e-11

"Compile a symbolic expression in the eigenvalue to a numeric function."
numfun(e, b) = Symbolics.build_function(e, b; expression = Val(false))

@testset "B.1 the printed mode shapes solve the beam eigenproblem" begin
    @variables b
    even = cosh(b * x) / cosh(b / 2) + cos(b * x) / cos(b / 2)
    odd = sinh(b * x) / sinh(b / 2) + sin(b * x) / sin(b / 2)

    @test zp(dx(even, 4) - b^4 * even)
    @test zp(dx(odd, 4) - b^4 * odd)

    ## The moment vanishes at both ends for any eigenvalue; this is built into the
    ## form of the modes and needs no frequency equation.
    for e in (even, odd), p in (1 // 2, -1 // 2)
        f = numfun(at(dx(e, 2), p), b)
        @test all(abs(f(bv)) < TOLF for bv in (1.3, 2.7, 4.1, 6.9))
    end

    ## The shear condition is what selects the eigenvalues, and it splits by parity.
    fe = numfun(at(dx(even, 3), 1 // 2) / b^3 - (tan(b / 2) + tanh(b / 2)), b)
    fo = numfun(at(dx(odd, 3), 1 // 2) / b^3 * tan(b / 2) * tanh(b / 2) -
        (tan(b / 2) - tanh(b / 2)), b)
    @test all(abs(fe(bv)) < TOLF for bv in (1.3, 2.7, 4.1, 6.9))
    @test all(abs(fo(bv)) < TOLF for bv in (1.3, 2.7, 4.1, 6.9))
    ## Control: the two conditions are genuinely different.
    fd = numfun(at(dx(even, 3), 1 // 2) - at(dx(odd, 3), 1 // 2), b)
    @test abs(fd(2.7)) > 1e-6

    ## Reflection parity of the two families. Symbolics does not reduce cosh(-u)
    ## to cosh(u), so these are evaluated numerically; the residual sits at

```

```

## round-off while the controls elsewhere are of order one.
pe = Symbolics.build_function(substitute(even, Dict(x => -x)) - even, b, x;
                             expression = Val(false))
po = Symbolics.build_function(substitute(odd, Dict(x => -x)) + odd, b, x;
                             expression = Val(false))
@test all(abs(pe(bv, xv)) < TOLF for bv in (1.3, 4.1), xv in (-0.4, 0.17, 0.5))
@test all(abs(po(bv, xv)) < TOLF for bv in (1.3, 4.1), xv in (-0.4, 0.17, 0.5))
end

@testset "B.1 the rigid modes are normalised" begin
    ## W0 = 1 and W1 = sqrt(12) x are the L2-orthonormal rigid-body pair, and the
    ## factor sqrt(12) is exactly what the normalisation fixes.
    @test zp(raftint((1 + 0 * x) * (1 + 0 * x)) - 1)
    @test zp(raftint(12 * x * x) - 1)
    @test zp(raftint((1 + 0 * x) * sqrt(12) * x))
    @test zp(dx(1 + 0 * x, 4)) && zp(dx(sqrt(12) * x, 4))
end

@testset "B.1 the printed modes are the textbook free-free modes" begin
    ## Textbook form on [0,1], with the frequency equation cos(b) cosh(b) = 1.
    freq(bv) = cos(bv) * cosh(bv) - 1
    function bisect(lo, hi)
        lo = BigFloat(lo); hi = BigFloat(hi)
        for _ in 1:400
            mid = (lo + hi) / 2
            freq(lo) * freq(mid) <= 0 ? (hi = mid) : (lo = mid)
        end
        (lo + hi) / 2
    end
    textbook(bv, s) = cosh(bv * s) + cos(bv * s) -
        ((cosh(bv) - cos(bv)) / (sinh(bv) - sin(bv))) * (sinh(bv * s) + sin(bv * s))
    centred_even(bv, u) = cosh(bv * u) / cosh(bv / 2) + cos(bv * u) / cos(bv / 2)
    centred_odd(bv, u) = sinh(bv * u) / sinh(bv / 2) + sin(bv * u) / sin(bv / 2)

    ## The two half-angle conditions factor the textbook frequency equation, so
    ## they select exactly the same eigenvalues.
    for bv in (BigFloat(1.7), BigFloat(5.3), BigFloat(9.1), BigFloat(12.4))
        A = sin(bv / 2) * cosh(bv / 2) + cos(bv / 2) * sinh(bv / 2)
        B = sin(bv / 2) * cosh(bv / 2) - cos(bv / 2) * sinh(bv / 2)
        @test abs(freq(bv) + 2 * A * B) < BigFloat(10)^-35
    end

    ## Shifting the centred mode by half a raft length reproduces the textbook
    ## shape. The odd modes differ by an overall sign, which cancels because a
    ## sign on W_n flips the modal amplitude and the radiation coefficient together.
    for (bv, shape) in ((bisect(4.0, 5.5), centred_even), (bisect(7.0, 8.5), centred_odd),
        (bisect(10.0, 11.5), centred_even), (bisect(13.5, 15.0), centred_odd))
        s0 = BigFloat(3) / 7
        c = textbook(bv, s0) / shape(bv, s0 - BigFloat(1) / 2)
        grid = range(BigFloat(0), BigFloat(1); length = 41)
        dev = maximum(abs(textbook(bv, s) - c * shape(bv, s - BigFloat(1) / 2)) for s in grid)
        scale = maximum(abs(textbook(bv, s)) for s in grid)
        @test dev / scale < 1e-30
        @test abs(abs(c) - 1) < 1e-30
    end

```

```

end
end

@testset "B.1 four integrations by parts (B7)" begin
    ## The identity is bilinear, so verifying it on generic polynomials verifies it.
    @variables w0 w1 w2 w3 w4 w5 w6 w7 e0 e1 e2 e3 e4 e5 e6 e7
    W = w0 + w1 * x + w2 * x^2 + w3 * x^3 + w4 * x^4 + w5 * x^5 + w6 * x^6 + w7 * x^7
    E = e0 + e1 * x + e2 * x^2 + e3 * x^3 + e4 * x^4 + e5 * x^5 + e6 * x^6 + e7 * x^7
    boundary(f, g) = (h = f * dx(g, 3) - dx(f, 1) * dx(g, 2) + dx(f, 2) * dx(g, 1) - dx(f, 3) * g;
        Symbolics.simplify(at(h, 1 // 2) - at(h, -1 // 2)))
    @test zp(raftint(W * dx(E, 4)) - raftint(dx(W, 4) * E) - boundary(W, E))

    ## Control: the identity needs all four boundary terms.
    partial(f, g) = (h = f * dx(g, 3) - dx(f, 1) * dx(g, 2) - dx(f, 3) * g;
        Symbolics.simplify(at(h, 1 // 2) - at(h, -1 // 2)))
    @test !zp(raftint(W * dx(E, 4)) - raftint(dx(W, 4) * E) - partial(W, E))
end

@testset "B.1 the capillary endpoint map (B8)" begin
    ## Of the four boundary terms only the one carrying eta''' survives: the
    ## basis satisfies the free-free conditions, and the raft moment vanishes at
    ## both ends. The shear condition then fixes what is left. The operator K
    ## appears on both sides of the edge condition and cancels, leaving
    ## kappa eta'''(+1/2) = +- (Lambda/We) eta_x(+1/2).
    @variables Lam Wen exp_r exp_l Wm_r Wm_l
    shear_right = (Lam / Wen) * exp_r
    shear_left = -(Lam / Wen) * exp_l
    Ksig = Wm_r * exp_r + Wm_l * exp_l
    @test zp((Wm_r * shear_right - Wm_l * shear_left) - (Lam / Wen) * Ksig)
    ## Control: equal signs at the two edges would give a difference, not a sum.
    @test !zp((Wm_r * (Lam / Wen) * exp_r - Wm_l * (Lam / Wen) * exp_l) - (Lam / Wen) * Ksig)
end

```

Appendix B.3: the zero-thrust condition

The starting point is the mechanical power the actuator delivers to the raft. Orthonormality of the basis carries it into the modal amplitudes, the cycle-average identity of Appendix A turns it into a real part, and the symmetry of the two parity blocks separates the reactive forces from the radiated power.

```

const TOL = BigFloat(10)^-40
nz(A) = maximum(abs.(A)) < TOL
rmat(n) = BigFloat.(rand(-9:9, n, n))
rvec(n) = BigFloat.(rand(-9:9, n))
cvec(n) = Complex{BigFloat}.(rand(-9:9, n)) .+ im .* Complex{BigFloat}.(rand(-9:9, n))
sym(A) = (A + transpose(A)) / 2
anti(A) = (A - transpose(A)) / 2
trials(f, n = 8) = all(f(k) for k in 1:n)

## An orthonormal real basis on the raft, built by Gram-Schmidt on the monomials.
## Only orthonormality is used below, exactly as in the paper.
function polyint(c)
    s = BigFloat(0)
    for k in 0:(length(c) - 1)
        iseven(k) && (s += c[k + 1] * 2 / ((k + 1) * BigFloat(2)^(k + 1)))
    end
end

```

```

    end
    s
end
function polymul(a, b)
    c = zeros(BigFloat, length(a) + length(b) - 1)
    for i in eachindex(a), j in eachindex(b)
        c[i + j - 1] += a[i] * b[j]
    end
    c
end
function orthonormal_basis(N)
    B = Vector{Vector{BigFloat}}{ }()
    for k in 0:(N - 1)
        v = vcat(BigFloat[i == k ? 1 : 0 for i in 0:k], zeros(BigFloat, N - k - 1))
        for w in B
            v -= polyint(polymul(v, w)) .* w
        end
        v ./= sqrt(polyint(polymul(v, v)))
        push!(B, v)
    end
    B
end

@testset "B.3 from the actuator power to the modal amplitudes" begin
    W = orthonormal_basis(4)
    @test all(abs(polyint(polymul(W[m], W[n]))) - (m == n ? 1 : 0)) < TOL
        for m in 1:4, n in 1:4)

    ## The cycle average of the physical integral, computed directly. The
    ## integrand is a trigonometric polynomial of degree two, so the trapezoid
    ## rule on one period is exact.
    function cycle_average(qb, fb, basis; N = 16, sgn = +1)
        step = BigFloat(2) * big(pi) / N
        acc = BigFloat(0)
        for m in 0:(N - 1)
            e = exp(im * m * step)
            etadot = sum(real(im * qb[n] * e) .* basis[n] for n in eachindex(qb))
            load = sum(real(fb[n] * e) .* basis[n] for n in eachindex(fb))
            acc += sgn * polyint(polymul(load, etadot))
        end
        acc / N
    end
    modal(qb, fb) = real(transpose(conj(fb)) * (im .* qb)) / 2

    ## Orthonormality is the only thing that carries the physical integral into
    ## the modal amplitudes.
    @test trials(k -> (qb = cvec(4); fb = cvec(4);
        abs(cycle_average(qb, fb, W) - modal(qb, fb)) < TOL))

    ## Control: a basis that is not orthonormal breaks the correspondence.
    skewed = [W[1], W[1] .+ W[2]]
    @test !trials(k -> (qb = cvec(2); fb = cvec(2);
        abs(cycle_average(qb, fb, skewed) - modal(qb, fb)) < TOL), 4)

    ## The actuator load acts downward while the elevation is measured upward, so

```

```

## the power delivered to the raft is -f eta_t. Eliminating the forcing with
## M q = -f cancels that sign and gives the form printed in (B17).
@test trials(k -> (qb = cvec(4); M = sym(rmat(4)) .+ im .* sym(rmat(4)); fb = -(M * qb);
    abs(cycle_average(qb, fb, W; sgn = -1) -
        real(transpose(conj(M * qb)) * (im .* qb)) / 2) < TOL))

## Control: the opposite convention returns the same expression with the
## wrong sign, which would make the radiated power negative.
@test trials(k -> (qb = cvec(4); M = sym(rmat(4)) .+ im .* sym(rmat(4)); fb = -(M * qb);
    abs(cycle_average(qb, fb, W; sgn = +1) +
        real(transpose(conj(M * qb)) * (im .* qb)) / 2) < TOL))

end

@testset "B.3 the cycle-average identity of Appendix A" begin
    ## The average of a product of two real harmonic signals is half the real
    ## part of one amplitude times the conjugate of the other.
    function average(ah, bh; N = 16)
        step = BigFloat(2) * big(pi) / N
        s = BigFloat(0)
        for m in 0:(N - 1)
            t = m * step
            s += real(ah * exp(im * t)) * real(bh * exp(im * t))
        end
        s / N
    end
    @test trials(k -> (ah = cvec(1)[1]; bh = cvec(1)[1];
        abs(average(ah, bh) - real(ah * conj(bh)) / 2) < TOL))
    ## It is the real part and not the modulus: the modulus carries no phase, so
    ## it cannot distinguish a load that does work from one that does not.
    @test !trials(k -> (ah = cvec(1)[1]; bh = cvec(1)[1];
        abs(average(ah, bh) - abs(ah) * abs(bh) / 2) < TOL))
    ## A load a quarter cycle out of phase with the velocity delivers no mean
    ## power however large its amplitude, while the modulus reports the same
    ## value in phase, in quadrature and in antiphase.
    for (phase, expected) in ((BigFloat(0), BigFloat(1) / 2),
        (big(pi) / 2, BigFloat(0)),
        (big(pi), -BigFloat(1) / 2))
        bh = exp(im * phase)
        @test abs(real(conj(bh)) / 2 - expected) < TOL
        @test abs(abs(conj(bh)) / 2 - BigFloat(1) / 2) < TOL
    end
end

@testset "B.3 the reactive block carries no mean power (B17)" begin
    power(H, Y, q) = real(transpose(conj((H .+ im .* Y) * q)) * (im .* q)) / 2
    quad(A, q) = transpose(conj(q)) * (A * q) / 2

    ## A real symmetric matrix gives a real quadratic form, a real antisymmetric
    ## one gives a purely imaginary form. This is the only fact used.
    @test trials(k -> (A = sym(rmat(4)); q = cvec(4); abs(imag(quad(A, q))) < TOL))
    @test trials(k -> (A = anti(rmat(4)); q = cvec(4); abs(real(quad(A, q))) < TOL))

    ## With both blocks symmetric the reactive term is i times a real number and
    ## is discarded by the real part.
    @test trials(k -> (H = sym(rmat(4)); Y = sym(rmat(4)); q = cvec(4);

```

```

        abs(power(H, Y, q) - real(quad(Y, q))) < TOL &&
        abs(imag(quad(Y, q))) < TOL))
## Control: without the symmetry of H the step fails. The appendix uses this
## hypothesis without stating it.
@test trials(k -> (H = rmat(4); Y = sym(rmat(4)); q = cvec(4);
        abs(power(H, Y, q) - real(quad(Y, q))) > TOL))
## The antisymmetric part of H leaves a real mean power that no radiated wave
## carries away.
@test trials(k -> (H = rmat(4); Y = sym(rmat(4)); q = cvec(4);
        abs(power(H, Y, q) -
        (real(quad(Y, q)) + imag(quad(anti(H), q)))) < TOL))
end

@testset "B.3 the radiation block has rank one (B18) and (B19)" begin
    J = BigFloat(7) / 3
    ## Equating the input and the radiated power for every modal displacement
    ## determines the matrix, and the factor two cancels the half of (B17).
    @test trials(k -> (a = cvec(4); Y = 2J .* (conj(a) * transpose(a)); q = cvec(4);
        abs(transpose(conj(q)) * (Y * q) / 2 - J * abs2(transpose(a) * q)) < TOL))
    ## A real Y forces every product of radiation coefficients to be real, which
    ## is the statement that they share one phase.
    @test trials(k -> (c = rvec(4); th = BigFloat(rand()) * 6; a = exp(im * th) .* c;
        Y = 2J .* (conj(a) * transpose(a));
        nz(imag.(Y)) && nz(Y - transpose(Y))))
    ## Control: coefficients without a common phase give a complex Y.
    @test trials(k -> (a = cvec(4); !nz(imag.(2J .* (conj(a) * transpose(a)))))
end

@testset "B.3 the transfer row and the interference matrix" begin
    ## Sherman-Morrison applied to the rank-one radiation term.
    @test trials(k -> begin
        H = rmat(4); c = rvec(4); gam = BigFloat(5) / 2; th = BigFloat(3) / 4
        M = H .+ im * gam .* (c * transpose(c))
        direct = -transpose(exp(im * th) .* c) * inv(M)
        printed = -exp(im * th) .* (transpose(c) * inv(H)) ./
            (1 + im * gam * (transpose(c) * inv(H) * c))
        nz(direct .- printed)
    end)
    ## The numerator is a real row and the denominator a single complex scalar,
    ## so every entry of the transfer row shares one phase.
    @test trials(k -> begin
        H = rmat(4); c = rvec(4); gam = BigFloat(5) / 2
        M = H .+ im * gam .* (c * transpose(c))
        r = -transpose(exp(im * BigFloat(3) / 4) .* c) * inv(M)
        nz(imag.(r ./ exp(im * angle(r[1]))))
    end)
    ## The interference matrix is then a real outer product scaled by the cosine
    ## of the phase difference between the two blocks.
    @test trials(k -> begin
        de = BigFloat(rand()) * 6; dd = BigFloat(rand()) * 6
        be = rvec(3); bo = rvec(3)
        re = exp(im * de) .* transpose(be); ro = exp(im * dd) .* transpose(bo)
        nz(real.(transpose(re) * conj(ro)) .- cos(de - dd) .* (be * transpose(bo)))
    end)
end

```

```

end)
## The forcing vector must be real for the thrust to factor this way.
@test trials(k -> begin
  re = transpose(cvec(3)); ro = transpose(cvec(3))
  fe = rvec(3); fo = rvec(3)
  G = real.(transpose(re) * conj(ro))
  abs(real((re * fe)[1] * conj((ro * fo)[1])) - (transpose(fe) * G * fo)) < TOL
end)
@test !trials(k -> begin
  re = transpose(cvec(3)); ro = transpose(cvec(3))
  fc = cvec(3); fo = rvec(3)
  G = real.(transpose(re) * conj(ro))
  abs(real((re * fc)[1] * conj((ro * fo)[1])) - real(transpose(fc) * G * fo)) < TOL
end)
end

@testset "B.3 the determinant lemma and the stiffness polynomial" begin
  @test trials(k -> begin
    H = rmat(4); c = rvec(4); gam = BigFloat(5) / 2
    M = H .+ im * gam .* (c * transpose(c))
    adjH = det(H) .* inv(H)
    abs(det(M) - (det(H) + im * gam * (transpose(c) * adjH * c))) < TOL
  end)
  ## With the determinant of the reactive block cancelled, the response to a
  ## real forcing is a real row divided by the block determinant, so the phase
  ## of the response is carried entirely by that determinant.
  @test trials(k -> begin
    H = rmat(4); c = rvec(4); gam = BigFloat(5) / 2; th = BigFloat(3) / 4
    M = H .+ im * gam .* (c * transpose(c))
    adjH = det(H) .* inv(H)
    P = det(H) + im * gam * (transpose(c) * adjH * c)
    f = rvec(4)
    abs(transpose(exp(im * th) .* c) * inv(M) * f -
      exp(im * th) * (transpose(c) * adjH * f) / P) < TOL
  end)
  ## The quarter-cycle condition on the two blocks is the vanishing of the
  ## real polynomial the paper assembles from the two determinants.
  @test trials(k -> begin
    the = BigFloat(rand()) * 6; tho = BigFloat(rand()) * 6
    Pe = Complex{BigFloat}(rand(1:9), rand(1:9))
    Po = Complex{BigFloat}(rand(1:9), rand(1:9))
    abs(cos((the - angle(Pe)) - (tho - angle(Po))) -
      real(exp(im * (the - tho)) * conj(Pe) * Po) / (abs(Pe) * abs(Po))) < TOL
  end)
end

@testset "B.3 the degree of the stiffness polynomial" begin
  @variables k
  function adjugate(A)
    n = size(A, 1)
    [Symbolics.expand((-1)^(i + j) * det(A[setdiff(1:n, j), setdiff(1:n, i)]))
     for i in 1:n, j in 1:n]
  end
end

```

```

degree(p) = (p = Symbolics.expand(p); m = 0;
    for d in 0:14
        iszero(Symbolics.coeff(p, k^d)) || (m = d)
    end; m)
function block(ne, gam)
    b4 = [0; rand(2:9, ne - 1)]
    H0 = Rational{Int}.(rand(-5:5, ne, ne))
    c = Rational{Int}.(rand(-4:4, ne))
    Hk = H0 .+ k .* Diagonal(b4)
    (Symbolics.expand(det(Hk)), Symbolics.expand(gam * (transpose(c) * adjugate(Hk) * c)))
end
for N in (4, 6, 8)
    ne = N ÷ 2
    ## Each parity block holds one rigid mode and the rest elastic.
    @test ne - 1 == div(N - 2, 2)
    Ae, Be = block(ne, 2 // 1)
    Ao, Bo = block(ne, 3 // 1)
    @test degree(Ae) <= div(N - 2, 2)
    @test degree(Be) <= div(N - 2, 2)
    for dth in (0.0, 1 / 3)
        pol = Symbolics.expand(cos(dth) * (Ae * Ao + Be * Bo) -
            sin(dth) * (Ae * Bo - Be * Ao))
        @test degree(pol) <= N - 2
    end
end

## Without hydrodynamic coupling the blocks are diagonal and the roots are
## the dry resonances of the elastic modes. The rigid mode contributes none.
b4 = [0 // 1, 500 // 1, 3803 // 1]
dry = Symbolics.expand(prod(k * bi - 1 for bi in b4))
@test all(iszero(Symbolics.value(Symbolics.simplify(substitute(dry, Dict(k => 1 // bi)))))
    for bi in b4 if bi != 0)
@test !iszero(Symbolics.value(Symbolics.simplify(substitute(dry, Dict(k => 0)))))
end

@testset "the stiffnesses quoted in Section 2.6" begin
    freq(bv) = cos(bv) * cosh(bv) - 1
    function bisect(lo, hi)
        lo = BigFloat(lo); hi = BigFloat(hi)
        for _ in 1:400
            mid = (lo + hi) / 2
            freq(lo) * freq(mid) <= 0 ? (hi = mid) : (lo = mid)
        end
        (lo + hi) / 2
    end
    for (lo, hi, quoted) in ((4.0, 5.5, 1.998e-3), (7.0, 8.5, 2.629e-4), (10.0, 11.5, 6.841e-5))
        bv = bisect(lo, hi)
        @test abs(Float64(1 / bv^4) - quoted) / quoted < 5e-4
    end
    ## The first and third elastic modes are even, the second is odd.
    @test abs(tan(bisect(4.0, 5.5) / 2) + tanh(bisect(4.0, 5.5) / 2)) < BigFloat(10)^-40
    @test abs(tan(bisect(7.0, 8.5) / 2) - tanh(bisect(7.0, 8.5) / 2)) < BigFloat(10)^-40
    @test abs(tan(bisect(10.0, 11.5) / 2) + tanh(bisect(10.0, 11.5) / 2)) < BigFloat(10)^-40
end

```

```
end
```

Appendix C: the uncoupled rigid limit

With the fluid coupling removed and the raft rigid, only the two rigid modes respond, and the motor position that nulls one end of the raft can be found in closed form.

```
@testset "C.1 the motor position that nulls one raft end" begin
    @variables X q0 s12
    ## s12 stands for the square root of twelve; its square is reduced exactly
    ## rather than in floating point.
    reduce12(e) = Symbolics.expand(substitute(Symbolics.expand(e), Dict(s12^2 => 12)))
    q1 = s12 * X * q0
    right = q0 + s12 / 2 * q1
    left = q0 - s12 / 2 * q1
    @test isequal(Symbolics.value(Symbolics.simplify(reduce12(right - q0 * (1 + 6X)))), 0)
    @test isequal(Symbolics.value(Symbolics.simplify(reduce12(left - q0 * (1 - 6X)))), 0)
    @test isequal(Symbolics.value(Symbolics.simplify(
        reduce12(substitute(right, Dict(X => -1 // 6, q0 => 1))))) , 0)
    @test isequal(Symbolics.value(Symbolics.simplify(
        reduce12(substitute(left, Dict(X => 1 // 6, q0 => 1))))) , 0)
    ## Control: the raft end does not null either boundary amplitude.
    @test !isequal(Symbolics.value(Symbolics.simplify(
        reduce12(substitute(right, Dict(X => -1 // 2, q0 => 1))))) , 0)
end
```